

Introduction to Android

Unit 1 – Theory Assignment

Name	Tarun Yadav
Roll No.	2401350015
Course	B.Tech CSE (FSD)
Subject	App development

Q1. What is Android? Explain its major features and applications.

Android is an open-source, Linux-kernel-based operating system developed by Google, designed primarily for touchscreen mobile devices such as smartphones and tablets, and later extended to wearables, TVs, and embedded systems. It provides a complete software stack for devices — an operating system, middleware, and a set of built-in applications — along with a rich set of APIs that developers use to build third-party applications, typically written in Java, Kotlin, or C++ (via the NDK).

Major Features of Android:

- **Open Source:** Based on the Android Open Source Project (AOSP), allowing manufacturers and developers to customize it freely.
- **Layered Architecture:** Consists of the Linux kernel, Hardware Abstraction Layer, native libraries, Android Runtime (ART), and the application framework.
- **Multitasking Support:** Allows multiple applications to run concurrently through processes and services.
- **Rich UI Framework:** Provides a wide range of UI components, layouts, and Material Design guidelines for building intuitive interfaces.
- **Connectivity:** Supports GSM/CDMA, Bluetooth, Wi-Fi, NFC, and other communication technologies.
- **Storage Options:** Offers SQLite databases, SharedPreferences, internal/external storage, and cloud-based storage.
- **Security:** Uses application sandboxing, permission-based access control, and Google Play Protect.
- **Google Services Integration:** Seamless access to Maps, Firebase, Google Play Services, and other Google APIs.

Applications of Android:

- Smartphones and tablets (the most common use case).
- Wearable devices through Wear OS (smartwatches, fitness bands).
- Smart TVs and set-top boxes via Android TV.
- In-car infotainment systems through Android Auto.
- IoT and embedded devices using Android Things / embedded variants.

Q2. What are Android Product Flavors? Explain with a suitable example.

Product Flavors are a feature of the Android Gradle build system that allow developers to create multiple different versions of the same application from a single codebase. Each flavor can have its own package name, resources, features, or configuration values, while still sharing the majority of the source code. Product flavors are usually combined with build types (like debug and release) to generate different build variants (for example, freeDebug, paidRelease, etc.).

Why they are used:

- To publish free and paid versions of the same app.
- To create different versions of an app for different clients (white-labelling).
- To target different environments, such as development, staging, and production servers.

Example (in app-level build.gradle):

```
android {  
    flavorDimensions "version"  
    productFlavors {  
        free {  
            applicationId "com.example.app.free"  
        }  
        paid {  
            applicationId "com.example.app.paid"  
        }  
    }  
}
```

Here, the free flavor could show ads and disable premium features, while the paid flavor could enable premium features, all built from the same project.

Q3. Explain the major functions of the Android Operating System.

The Android Operating System, being built on top of the Linux kernel, is responsible for managing the hardware and software resources of the device and providing common services that applications rely on. Its major functions include:

- **Process Management:** Creating, scheduling, and terminating application processes, and managing their lifecycle states.
- **Memory Management:** Allocating memory to running applications and reclaiming it through garbage collection and low-memory killer mechanisms.
- **Device Driver Management:** Managing drivers for the display, camera, Wi-Fi, Bluetooth, audio, and other hardware components.
- **Security Management:** Enforcing application sandboxing so that each app runs in its own process with a unique Linux user ID, and controlling access through permissions.
- **Network Management:** Handling network connectivity (mobile data, Wi-Fi) and providing APIs for communication over these networks.
- **Resource Management:** Allocating and managing storage, battery, and CPU resources efficiently among applications.
- **Application Framework Services:** Providing core services such as the Activity Manager, Window Manager, Content Providers, and Notification Manager that applications use.
- **User Interface Management:** Rendering UI elements and handling touch, gesture, and sensor-based user input.

Q4. What is Gradle? Explain its role in Android application development.

Gradle is an open-source, automation-based build tool that is used as the official build system for Android application development. It uses a Groovy or Kotlin DSL (Domain Specific Language) to describe the build configuration in files such as `build.gradle` (or `build.gradle.kts`), instead of relying on rigid XML configuration.

Role of Gradle in Android Development:

- **Compiling Code:** Compiles Java/Kotlin source code along with resources into a runnable APK or AAB (Android App Bundle).
- **Dependency Management:** Automatically downloads and manages external libraries from repositories like Maven Central and Google's Maven repository.
- **Build Variants:** Supports creation of multiple build types (debug/release) and product flavors from a single project.
- **Resource Merging:** Merges resources, manifests, and code from libraries and modules into the final application.
- **Task Automation:** Automates repetitive tasks such as testing, code linting, packaging, and signing the APK.
- **Multi-Module Support:** Manages large projects that are split into multiple modules/libraries.
- **Customizability:** Allows developers to write custom build logic and plugins for specialized workflows (e.g., CI/CD pipelines).

Q5. Differentiate between Groovy and Kotlin DSL in Android projects.

Aspect	Groovy DSL	Kotlin DSL
File Extension	build.gradle	build.gradle.kts
Language	Based on the dynamically-typed Groovy language	Based on the statically-typed Kotlin language
Type Safety	Weakly typed; errors often appear only at build/runtime	Strongly typed; many errors are caught at compile time / by the IDE
IDE Support	Basic auto-completion in Android Studio	Superior auto-completion, refactoring, and navigation support
Syntax Style	More concise and script-like, uses closures	More verbose, closer to standard Kotlin code syntax
Learning Curve	Easier for beginners, historically the default	Slightly steeper if unfamiliar with Kotlin, but consistent with app code
Build Speed	Generally faster to configure/parse for smaller projects	Can be slightly slower to configure, though gap has narrowed
Industry Trend	Legacy/older projects still widely use this	Recommended by Google for new projects going forward

Q6. Explain the purpose of the following Android project files/folders: java, res, AndroidManifest.xml, build.gradle, settings.gradle.

java folder: This folder contains all the source code of the application, written in Java or Kotlin. It holds activities, fragments, adapters, models, and other classes that define the app's logic and behaviour.

res folder (resources): This folder stores all the non-code resources used by the application, organized into sub-folders such as layout (XML UI layouts), drawable (images/icons), values (strings, colors, styles, dimensions), mipmap (launcher icons), and menu (menu definitions). Keeping resources separate from code allows easy support for different screen sizes, languages, and themes.

AndroidManifest.xml: This is a mandatory XML file present in every Android project that describes essential information about the app to the Android build tools, the OS, and the Google Play Store. It declares the app's package name, components (activities, services, broadcast receivers, content providers), required permissions, minimum/target SDK versions, and the app's icon and theme.

build.gradle: This file contains the build configuration for the project and its modules. The project-level build.gradle configures settings common to all modules (like repository locations), while the app/module-level build.gradle specifies the applicationId, SDK versions, dependencies, build types, and product flavors specific to that module.

settings.gradle: This file defines which modules are included in the overall project build. For a project with multiple modules (e.g., an app module and a library module), settings.gradle lists all of them so that Gradle knows to include them when building the project.

Q7. What is Minimum SDK? Why is it required while creating an Android project?

The Minimum SDK (`minSdkVersion`) is a configuration value, set in the app's `build.gradle` file, that specifies the lowest Android API level on which the application is guaranteed to run. It defines the minimum version of the Android operating system a user's device must have installed in order to install and run the app from the Google Play Store.

Why it is required:

- **Compatibility Control:** It prevents the app from being installed on devices running an Android version too old to support the APIs the app depends on, avoiding crashes.
- **API Availability:** Newer Android APIs and features are not available on older OS versions; the `minSdkVersion` tells the compiler which APIs can safely be used without extra checks.
- **Market Reach vs. Features Trade-off:** Setting a lower `minSdkVersion` increases the number of devices that can install the app but may restrict the developer to older, more limited APIs; a higher `minSdkVersion` allows use of modern APIs but reduces the number of compatible devices.
- **Play Store Filtering:** The Google Play Store uses this value to automatically hide the app from users whose device OS version is below the specified minimum.
- **Build-Time Warnings:** Android Studio uses `minSdkVersion` to warn developers at compile time if they use an API that is not available on the minimum supported version.

Q8. Explain any five commonly used Android UI components and their uses.

1. **TextView:** A UI component used to display non-editable text to the user, such as labels, headings, or descriptive content. It supports styling attributes like text size, color, and font.
2. **EditText:** An editable text field that allows users to input text, numbers, passwords, or other data. It is commonly used in forms, login screens, and search bars, and supports input type restrictions (e.g., numeric, email).
3. **Button:** A clickable UI element that triggers an action when pressed, such as submitting a form or navigating to another screen. It can display text or an icon and responds to click listeners in code.
4. **ImageView:** A component used to display images, icons, or drawable resources within the app's UI. It supports scaling types to control how the image fits within its bounds.
5. **RecyclerView:** An advanced and efficient component used to display large or scrollable lists/grids of data, such as a list of contacts or products. It reuses (recycles) item views as the user scrolls, making it more memory-efficient than the older ListView.

Q9. What is a Layout in Android? Explain ConstraintLayout and LinearLayout.

A Layout in Android is a ViewGroup — an invisible container — that defines the visual structure and arrangement of UI components (Views) on the screen, such as their position, size, and how they relate to each other. Layouts are usually defined declaratively using XML files inside the res/layout folder, and they form the root of the view hierarchy that is inflated at runtime.

LinearLayout:

LinearLayout arranges its child views in a single direction, either horizontally or vertically, one after another, as specified by the android:orientation attribute. Each child is placed sequentially in the given direction, and the layout_weight attribute can be used to distribute remaining space proportionally among children. It is simple to use but can become inefficient and hard to manage for complex, nested UI designs.

ConstraintLayout:

ConstraintLayout is a more flexible and powerful layout that allows positioning of views relative to each other or to the parent layout using a system of constraints (e.g., constraining a view's left edge to the parent's left edge, or a view's top to another view's bottom). It helps create complex, responsive UI designs with a flat view hierarchy (avoiding deep nesting), which improves rendering performance. It is the recommended default layout in modern Android Studio projects and works well with the visual Layout Editor.

Q10. Differentiate between CheckBox, RadioButton, Spinner, and Switch with suitable examples.

Component	Selection Behaviour	Typical Use / Example
CheckBox	Allows selection of one or more options independently; can be checked/unchecked freely.	Selecting multiple hobbies: e.g., Reading, Sports, Music (any combination)
RadioButton	Allows selection of only one option from a group (mutually exclusive), grouped inside a RadioGroup.	Selecting gender or a single choice in a quiz: Male / Female
Spinner	A dropdown-style widget that shows one visible item at a time from a list; user taps to open and pick one.	Selecting a country or state from a long list of options.
Switch	A toggle control representing an on/off (binary) state, usually with immediate effect.	Turning Wi-Fi or Dark Mode ON/OFF in settings.